

Raspberry Pi

Puerto GPIO

Puerto GPIO

Introducción

Antes de comenzar a trabajar con el puerto GPIO, es importante tener presente las siguientes recomendaciones para no causar algún daño a la Raspberry Pi.

- La Raspberry Pi funciona a 3.3V y no tolera mucho los errores.
- Basta con que conectes 5V tranquilamente aun pin de tu Rasberry PI para que se dañe.
- La Raspberry Pi no cuenta con algún puerto o convertidor Analógico/Digital, ni a la inversa.



Puerto GPIO

Librerías y/o Paquetes Necesarios

Para hacer uso del puerto GPIO desde Python es necesario hacer uso del paquete Rpi.GPIO, lo instalamos con al siguientes instrucción.

```
sudo apt-get install python-rpi.gpio python3-rpi.gpio
```



Puerto GPIO

Ejemplos

Para verificar si podemos hacer uso de la librería RPi.GPIO lo haremos a través del siguiente ejemplo.

```
try:
    import RPi.GPIO as GPIO
except RuntimeError:
    print("Error al importar RPi.GPIO! talvez necesite permisos de superuuario")
else:
    print("Se puede hacer uso de RPi.GPIO")
```



Puerto GPIO

Modo de Trabajo

Para determinar el modo en que se trabajar con los pines desde Python es con la siguiente instrucción.

```
GPIO.setmode(GPIO.BOARD)
  # O
GPIO.setmode(GPIO.BCM)
```

Para saber en qué modalidad se está trabajando es con la siguiente función .

```
modo = GPIO.getmode()
```

Valor	Descripción
None	Desconocido o no configurado (None)
10	GPIO.BOARD
11	GPIO.BCM



Puerto GPIO

Ejemplo

Ahora mostraremos un ejemplo del uso de GPIO.getmode().

```
import RPi.GPIO as GPIO
modo = GPIO.getmode()
print("El modo actual es: ",modo)
GPIO.setmode(GPIO.BCM)
modo = GPIO.getmode()
print("El modo cambio a: ",modo)
```

Puerto GPIO

Funciones de los Pines

Para saber cómo se encuentra configurado un pin en específico ocupamos la siguiente instrucción.

```
func = GPIO.gpio_function(pin)
```

Valor	Descripción
-1	Desconocido o no configurado (GPIO.UNKNOWN)
0	Salida (GPIO.OUT)
1	Entrada(GPIO.IN)
40	Para comunicaciones Seriales UART (GPIO.SERIAL)
41	Comunicaciones SPI(GPIO.SPI)
42	Comunicaciones I2C (GPIO.I2C)
43	Funciones (GPIO.HARD_PWM)



Puerto GPIO

Funciones de los Pines

Para saber cómo se encuentra configurado un pin en específico ocupamos la siguiente instrucción.

```
func = GPIO.gpio_function(pin)
```

Valor	Descripción
-1	Desconocido o no configurado (GPIO.UNKNOWN)
0	Salida (GPIO.OUT)
1	Entrada(GPIO.IN)
40	Para comunicaciones Seriales UART (GPIO.SERIAL)
41	Comunicaciones SPI(GPIO.SPI)
42	Comunicaciones I2C (GPIO.I2C)
43	Funciones (GPIO.HARD_PWM)

Puerto GPIO

Ejemplo

El siguiente código nos muestra el valor que identifica la función con el que este configurado cada pin..

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setup(17, GPIO.IN)
GPIO.setup(27,GPIO.OUT)
func=GPIO.gpio_function(25)
print("El puerto 17 funciona como: ",func)
func=GPIO.gpio_function(17)
print("El puerto 17 funciona como: ",func)
func=GPIO.gpio_function(27)
print("El puierto 27 funciona como: ",func)
```

Puerto GPIO

Salidas Digitales

Para describir el uso de las salidas digitales de nuestra Raspberry Pi , lo haremos con un ejemplo, este ejemplo sencillo lo único que hará es encender un led por 3 segundos y al término de este tiempo apagarlo.

```
importRPi.GPIO as GPIO  #Importamos la Librería RPi,GPIO
import time             #Importamos la Librería time
GPIO.setwarnings(False) #Evitamos que nos Mande Advertencias
GPIO.setmode(GPIO.BCM)  #Definimos el modo de Trabajo BCM
GPIO.setup(17,GPIO.OUT) #Definimos el PIN 17 como salida
GPIO.output(17,True)    #Encendemos el Led
time.sleep(3)           #Hacemos una pausa de 3 segundos
GPIO.output(17,False)   #Apagamos el Led
GPIO.cleanup()          #Liberamos los recursos
```



Puerto GPIO

Salidas Digitales

A continuación realizaremos un ejemplo un poco más elaborado, haremos un programa para hacer parpadear el diodo Led.

```
importRPi.GPIO as GPIO
import time
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(17,GPIO.OUT)
try:
    while(True):
        GPIO.output(17,True)
        time.sleep(1)
        GPIO.output(17,False)
        time.sleep(1)
exceptKeyboardInterrupt:
    print("Programa termido por el Usuario")
finally:
    GPIO.cleanup()
```



Puerto GPIO

Salidas Digitales

Se puede trabajar con una serie de salidas en conjunto, esto es posible con el uso de listas, a continuación se realizara un ejemplo.

```
import RPi.GPIO as GPIO
import RPi.GPIO as GPIO
import time
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
lista=[24,16]
GPIO.setup(lista,GPIO.OUT)
try:
    while(True):
        GPIO.output(lista,(True, False))
        time.sleep(0.5)
        GPIO.output(lista,(False, True))
        time.sleep(0.5)
except KeyboardInterrupt:
    print("Programa terminado por el usuario")
finally:
    GPIO.cleanup()
```



Puerto GPIO

Entradas Digitales.

RPi.GPIO tiene la capacidad de configurar resistencias Pull up / Pull down desde software de la siguiente manera.

```
GPIO.setup(canal, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(canal, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
```

Puerto GPIO

Entradas Digitales.

En el siguiente ejemplo estamos indicando que se trabajar con el pin GPIO 7 como entrada y trabajara con una resistencia Pull Up.

```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BCM)
GPIO.setup(7, GPIO.IN, pull_up_down=GPIO.PUD_UP)
Status = GPIO.input(7)
try:
    while True:
        Status  = GPIO.input(7)
        if Status  == False:
            print("Boton pulsado.")
            time.sleep(0.2)
except KeyboardInterrupt:
    print("Programa Terminado por el Usuario")
finally:
    GPIO.cleanup()
```



Puerto GPIO

Cambios de estado (Flanco).

Existen dos maneras para poder detectar estos eventos mientras el programa está ocupado realizando otras tareas.

La primera forma para realizar esta tarea es mediante el uso de la función `wait_for_edge`(), esta función interrumpe la ejecución programa hasta que se detecta un cambio de estado o caduca un tiempo de espera dado;

Puerto GPIO

Cambios de estado (Flanco).

```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BCM)
GPIO.setup(25, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(12, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
try:
    pin1 = GPIO.wait_for_edge(25, GPIO.FALLING, timeout=50000)
    if pin1 == 25:
        print("Se presionó el pin No: ", pin)
    pin2 = GPIO.wait_for_edge(12,GPIO.RISING, timeout=50000)
    if pin2 == 12:
        print("Se presionó el pin No:", pin2)
except:
    print("Ocurrio un error")
finally:
    GPIO.cleanup()
    print("programa finalizado")
```



Puerto GPIO

Cambios de estado (Flanco).

La segunda forma para realizar esta tarea es mediante el uso de la función `add_event_detect()`, esta función realiza alguna tarea dada cuando detecta un cambio de estado.

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)

def evento_boton(canal):
    print("Presiono el boton: ", canal)
try:
    GPIO.setup(25, GPIO.IN, pull_up_down=GPIO.PUD_UP)
    GPIO.add_event_detect(25, GPIO.FALLING, callback=evento_boton)
    i=0
    while True:
        i=i + 1
        print(i)
        time.sleep(1)
except KeyboardInterrupt:
    print("Programa terminado por el usuario")
finally:
    GPIO.remove_event_detect(25)
    GPIO.cleanup()
    print("Programa terminado")
```



Puerto GPIO

Cambios de estado (Flanco).

Esta función también puede recibir un tercer parámetro, que es establecer un tiempo de espera para evitar el rebote evitando que la interrupción se active durante dicho periodo de espera.

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)

def evento_boton(canal):
    print("Presiono el boton: ", canal)
try:
    GPIO.setup(25, GPIO.IN, pull_up_down=GPIO.PUD_UP)
    GPIO.add_event_detect(25, GPIO.FALLING, callback=evento_boton, bouncetime=300)
    i=0
    while True:
        i=i + 1
        print(i)
        time.sleep(1)
except KeyboardInterrupt:
    print("Programa terminado por el usuario")
finally:
    GPIO.remove_event_detect(25)
    GPIO.cleanup()
```



Puerto GPIO

Cambios de estado (Flanco).

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)

def evento_boton(canal):
    print("Presiono el boton: ", canal)
def evento_boton2(canla):
    print("Presiono el segundo boton")
try:
    GPIO.setup(25, GPIO.IN, pull_up_down=GPIO.PUD_UP)
    GPIO.setup(12, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

    GPIO.add_event_detect(25, GPIO.FALLING)
    GPIO.add_event_detect(12, GPIO.RISING)
    GPIO.add_event_callback(25, evento_boton)
    GPIO.add_event_callback(12, evento_boton2)
    i=0
    while True:
        i=i + 1
        print(i)
        time.sleep(1)
except KeyboardInterrupt:
    print("Programa terminado por el usuario")
finally:
    GPIO.remove_event_detect(25)
    GPIO.remove_event_detect(12)
    GPIO.cleanup()
```

